Lecture Notes: TwinCAT 3

Interfaces and Polymorphism

March 14, 2026

Contents

1 Introduction to Interfaces 2

2 Implementing Interfaces 3

3 Why Use Interfaces? (Key Advantages) 5

4 Advanced Pattern: Dependency Injection 6

5 Practical Use Case: Persistent Event Logging 8

Quick Reference Card 10

1 Introduction to Interfaces

Interfaces represent a significant step into professional **Object-Oriented Programming (OOP)** in TwinCAT 3. While inheritance (using `EXTENDS`) focuses on sharing functionality, interfaces are a mechanism for **abstraction** and **polymorphism**.

- **Definition:** An interface specifies **what** a function block (FB) must do, but not **how** it does it. It acts as a "blueprint" or a "binding contract".
- **The Contract:** Any FB that implements an interface is legally bound to implement all the methods defined within that interface.
- **Analogy:** Think of a chessboard where every piece (King, Pawn) is an object. They might all implement a `Move()` interface, but the actual implementation of how they move is unique to each piece.

Aspect	Inheritance (EXTENDS)	Interface (IMPLEMENTS)
Purpose	Share functionality	Define contract
Contains	Code implementation	Method signatures only
Multiple	Single inheritance only	Multiple interfaces allowed
Focus	"How" things work	"What" must be done

Table 1: Inheritance vs. Interfaces

Example: Interface Definition

```
// Interface: I_LightTower
// Defines the contract for any light tower implementation
// Note: Interfaces contain ONLY method signatures - NO
    implementation code
INTERFACE I_LightTower
```

```
// Method: PowerOn
// Turns the light tower on or off
// Note: No code body - just the method signature
METHOD PowerOn
VAR_INPUT
    bEnable : BOOL;     // TRUE = power on, FALSE = power off
END_VAR
```

```
// Method: SetColor
// Sets the active color of the light tower
METHOD SetColor
VAR_INPUT
    eColor : E_Color;  // Enumeration: Red, Yellow, Green, Blue, etc.
END_VAR
```

```
// Method: EnableBuzzer
// Enables or disables the audible buzzer
METHOD EnableBuzzer
VAR_INPUT
    bEnable     : BOOL;     // TRUE = buzzer on, FALSE = buzzer off
```

```
    nFrequency : UINT;     // Buzzer frequency in Hz (optional parameter)
END_VAR
```

Note: Interface methods have **no code bodies**. They only declare the method name, return type, and parameters. The actual implementation is provided by the function blocks that implement the interface.

2 Implementing Interfaces

To use an interface, a function block must explicitly state that it follows that interface's contract.

- **Syntax:** Use the `IMPLEMENTS` keyword in the FB declaration.
- **Implementation Details:** The actual work (the code) is written inside the FB's methods. For example, one light tower might be controlled by simple 24V digital outputs, while another uses complex protocols like **EtherCAT** or **IO-Link**, yet both follow the same interface.
- **Multiple Interfaces:** Unlike inheritance—where an FB can only `EXTEND` one other FB—an FB can implement **as many interfaces as it wants**, separated by commas.

Example: Implementing an Interface

```
// Function Block: FB_Balluff_LightTower
// Implements the I_LightTower interface for Balluff hardware
// Uses IO-Link communication protocol
FUNCTION_BLOCK FB_Balluff_LightTower IMPLEMENTS I_LightTower
VAR
    // Internal variables specific to Balluff implementation
    bPowerState      : BOOL;            // Current power state
    eCurrentColor    : E_Color;         // Currently displayed color
    bBuzzerActive    : BOOL;            // Buzzer state
    stIOLinkData     : ST_IOLinkData;   // IO-Link communication structure
END_VAR
```

```
// Method Implementation: PowerOn
// Balluff-specific implementation of the PowerOn method
METHOD PowerOn
VAR_INPUT
    bEnable : BOOL;
END_VAR

// Balluff-specific IO-Link command to control power
bPowerState := bEnable;
stIOLinkData.nCommand := 16#01;  // Power control command
stIOLinkData.bValue := bEnable;

// Send command via IO-Link (implementation details hidden)
THIS^.SendIOLinkCommand();
```

```
// Method Implementation: SetColor
// Balluff-specific implementation of the SetColor method
```

```
METHOD SetColor
VAR_INPUT
    eColor : E_Color;
END_VAR

// Store the color and send to hardware
eCurrentColor := eColor;
stIOLinkData.nCommand := 16#02;  // Color control command

CASE eColor OF
    E_Color.Red:
        stIOLinkData.nValue := 1;
    E_Color.Yellow:
        stIOLinkData.nValue := 2;
    E_Color.Green:
        stIOLinkData.nValue := 3;
END_CASE

THIS^.SendIOLinkCommand();
```

```
// Method Implementation: EnableBuzzer
// Balluff-specific implementation of the EnableBuzzer method
METHOD EnableBuzzer
VAR_INPUT
    bEnable    : BOOL;
    nFrequency : UINT;
END_VAR

bBuzzerActive := bEnable;
stIOLinkData.nCommand := 16#03;  // Buzzer control command
stIOLinkData.bValue := bEnable;
stIOLinkData.nValue := nFrequency;

THIS^.SendIOLinkCommand();
```

```
// Alternative Implementation: FB_Sick_LightTower
// Same interface, completely different hardware and protocol
FUNCTION_BLOCK FB_Sick_LightTower IMPLEMENTS I_LightTower
VAR
    // Sick-specific variables (different from Balluff)
    bPowerState      : BOOL;
    eCurrentColor    : E_Color;
    stEtherCATFrame  : ST_EtherCATFrame;  // Uses EtherCAT instead of IO-Link
    nNodeAddress     : UINT;              // EtherCAT node address
END_VAR

// Methods would have completely different implementation
// but fulfill the same interface contract
```

Tip: When implementing multiple interfaces, separate them with commas: `FUNCTION_BLOCK FB_AdvancedTower IMPLEMENTS I_LightTower, I_Diagnostic, I_Configurable`

3 Why Use Interfaces? (Key Advantages)

Interfaces provide a layer of abstraction that allows developers to write code that is "ignorant of unnecessary details".

1. **Flexibility at Runtime:** You can declare an interface variable and assign different concrete FBs to it while the PLC is running. This allows for swapping hardware or logic dynamically.
2. **Modularization and Teamwork:** Different teams (e.g., Team A for machine logic and Team B for file logging) only need to agree on an interface "contract". They can then develop their parts independently without knowing the internal details of the other's code.
3. **Testability (Mocking):** Interfaces allow you to "mock" dependencies. For example, if a function block depends on a database, you can create a "mock" interface for testing that doesn't require a real database connection.

Example: Runtime Flexibility

```
// Program: MAIN
// Demonstrates polymorphism - same interface, different implementations
PROGRAM MAIN
VAR
    // Interface variable - can hold ANY FB that implements I_LightTower
    iActiveLightTower : I_LightTower;

    // Concrete implementations (different hardware manufacturers)
    fbBalluffTower     : FB_Balluff_LightTower;   // Balluff hardware
    fbSickTower        : FB_Sick_LightTower;      // Sick hardware
    fbSimulatedTower   : FB_Simulated_LightTower; // For testing without hardware

    // Selection variable
    nSelectedTower     : INT := 1;
END_VAR

// Assign the appropriate implementation based on configuration
// This can be changed at runtime!
CASE nSelectedTower OF
    1:  // Use Balluff hardware
        iActiveLightTower := fbBalluffTower;

    2:  // Use Sick hardware
        iActiveLightTower := fbSickTower;

    3:  // Use simulation for testing
        iActiveLightTower := fbSimulatedTower;
END_CASE

// Now use the interface - the calling code doesn't care which
// implementation is actually being used!
// This is POLYMORPHISM in action
iActiveLightTower.PowerOn(bEnable := TRUE);
iActiveLightTower.SetColor(eColor := E_Color.Green);
iActiveLightTower.EnableBuzzer(bEnable := FALSE, nFrequency := 0);
```

```
// Example: Switching hardware at runtime
// Scenario: Primary tower fails, switch to backup
VAR
    fbPrimaryTower   : FB_Balluff_LightTower;
    fbBackupTower    : FB_Sick_LightTower;
    iCurrentTower    : I_LightTower;
    bPrimaryFailed   : BOOL;
END_VAR

// Dynamic reassignment based on hardware status
IF bPrimaryFailed THEN
    iCurrentTower := fbBackupTower;     // Seamless switch to backup
ELSE
    iCurrentTower := fbPrimaryTower;    // Use primary
END_IF

// Code continues to work regardless of which tower is active
iCurrentTower.SetColor(eColor := E_Color.Red);
```

Warning: Always check that an interface reference is valid before using it. An unassigned interface reference will cause a runtime error if methods are called on it.

4 Advanced Pattern: Dependency Injection

The sources introduce **Dependency Injection**, a technique where an object receives its dependencies (other objects it needs) from the outside rather than creating them itself.

- **FB_init (The Constructor):** In TwinCAT, dependency injection is often handled via the `FB_init` method, which is called implicitly when an object is instantiated.
- **Mechanism:** You pass a reference to an interface into `FB_init` and store that reference locally within the FB. This decouples the "client" FB from the specific "service" implementation.

Aspect	Without Dependency Injection	With Dependency Injection
Coupling	Tight - FB creates its own dependencies	Loose - dependencies passed in
Testing	Difficult - real dependencies required	Easy - mock objects can be injected
Flexibility	Low - hard-coded implementations	High - swap implementations easily
Reusability	Limited	High

Table 2: Benefits of Dependency Injection

Example: Dependency Injection via FB_init

```
// Interface: I_PersistentEventStorage
// Contract for any event storage mechanism
INTERFACE I_PersistentEventStorage
```

```
// Method signature for storing an event
METHOD StoreEvent : BOOL
VAR_INPUT
    stEvent : REFERENCE TO ST_Event;  // Event to be stored
END_VAR
```

```
// Function Block: FB_EventLogger
// Uses dependency injection to receive its storage mechanism
FUNCTION_BLOCK FB_EventLogger
VAR CONSTANT
    MAXIMUM_BUFFER_SIZE : UDINT := 100;
END_VAR
VAR
    // Internal event buffer
    astEventBuffer : ARRAY[0..(MAXIMUM_BUFFER_SIZE - 1)] OF ST_Event;
    nCurrentIndex  : UDINT := 0;

    // INJECTED DEPENDENCY - stored as interface reference
    // This allows any storage implementation to be used
    iPersistentStorage : I_PersistentEventStorage;
    bStorageAvailable  : BOOL := FALSE;
END_VAR
```

```
// FB_init Method: Constructor for FB_EventLogger
// This is where dependency injection happens
// Called automatically when the FB is instantiated
METHOD FB_init : BOOL
VAR_INPUT
    bInitRetains : BOOL;  // Required parameter (system)
    bInCopyCode  : BOOL;  // Required parameter (system)

    // INJECTED DEPENDENCY: Reference to storage interface
    iStorage : I_PersistentEventStorage;
END_VAR

// Store the injected dependency for later use
// Now the EventLogger can use ANY storage implementation
iPersistentStorage := iStorage;

// Verify that a valid storage was provided
IF iPersistentStorage <> 0 THEN
    bStorageAvailable := TRUE;
ELSE
    bStorageAvailable := FALSE;
END_IF

FB_init := TRUE;
```

```
// Usage: Instantiating FB_EventLogger with dependency injection
// The storage implementation is "injected" at instantiation time
PROGRAM MAIN
VAR
    // Different storage implementations
    fbXmlStorage  : FB_XmlStorage;      // Stores events to XML file
    fbJsonStorage : FB_JsonStorage;     // Stores events to JSON file
    fbDbStorage   : FB_DatabaseStorage; // Stores events to database
    fbMockStorage : FB_MockStorage;     // For unit testing
```

```

    // Event logger with XML storage injected
    fbEventLoggerXml : FB_EventLogger(iStorage := fbXmlStorage);

    // Event logger with database storage injected
    fbEventLoggerDb  : FB_EventLogger(iStorage := fbDbStorage);

    // Event logger with mock storage for testing
    fbEventLoggerTest : FB_EventLogger(iStorage := fbMockStorage);
END_VAR
```

Tip: Dependency injection makes your code more testable. During testing, inject a mock storage that doesn't actually write to disk or database, allowing fast and isolated unit tests.

5 Practical Use Case: Persistent Event Logging

The tutorial refines an `EventLogger` to separate its core logic (managing an event array) from the logic of storing those events (XML, JSON, or Database).

- **Separation of Concerns:** The `EventLogger` doesn't care *how* an event is stored; it only cares that a storage service exists.
- **The Workflow:**
  1. Create an interface `I_PersistentEventStorage` with a `StoreEvent()` method.
  2. Create specific FBs (e.g., `FB_XmlStorage`) that implement this interface.
  3. Inject the storage FB into the `EventLogger` during instantiation.

Example: Calling the Interface Method

```
// Method: AddEvent (Public method of FB_EventLogger)
// Adds an event to the buffer and persists it via the injected
    storage
METHOD PUBLIC AddEvent : BOOL
VAR_INPUT
    nEventId     : UDINT;
    sEventText   : STRING(255);
    eSeverity    : TcEventSeverity;
END_VAR
VAR
    fbGetSystemTime : GETSYSTEMTIME;
    stFileTime      : T_FILETIME;
    bStoreResult    : BOOL;
END_VAR

// Check if buffer is full
IF THIS^.IsEventBufferFull() THEN
    AddEvent := FALSE;
    RETURN;
END_IF

// Populate the event structure
```

```
astEventBuffer[nCurrentIndex].nIdentity      := nEventId;
astEventBuffer[nCurrentIndex].sText          := sEventText;
astEventBuffer[nCurrentIndex].eEventSeverity := eSeverity;

// Get timestamp
fbGetSystemTime(timeLoDW => stFileTime.dwLowDateTime,
                timeHiDW => stFileTime.dwHighDateTime);
astEventBuffer[nCurrentIndex].dtTimestamp := FILETIME_TO_DT(stFileTime);

// ========================================================
// CALL THE INJECTED INTERFACE METHOD
// The EventLogger doesn't know or care HOW the event is stored
// It just calls the interface method - polymorphism handles the rest
// ========================================================
IF bStorageAvailable THEN
    // Call StoreEvent on whatever storage implementation was injected
    // Could be XML, JSON, Database, or Mock - we don't care here!
    bStoreResult := iPersistentStorage.StoreEvent(
        stEvent := astEventBuffer[nCurrentIndex]
    );

    IF NOT bStoreResult THEN
        // Handle storage failure (log warning, retry, etc.)
        // Event is still in buffer even if persistence fails
    END_IF
END_IF

// Update index
nCurrentIndex := nCurrentIndex + 1;

AddEvent := TRUE;
```

```
// Example Storage Implementation: FB_XmlStorage
// Implements I_PersistentEventStorage for XML file storage
FUNCTION_BLOCK FB_XmlStorage IMPLEMENTS I_PersistentEventStorage
VAR
    sFilePath     : STRING(255) := 'C:\EventLogs\events.xml';
    fbFileWrite   : FB_FileWrite;  // File writing function block
    bBusy         : BOOL;
END_VAR
```

```
// Implementation of StoreEvent for XML storage
METHOD StoreEvent : BOOL
VAR_INPUT
    stEvent : REFERENCE TO ST_Event;
END_VAR
VAR
    sXmlContent : STRING(1000);
END_VAR

// Format event as XML
sXmlContent := CONCAT('<Event>');
sXmlContent := CONCAT(sXmlContent, '<ID>');
sXmlContent := CONCAT(sXmlContent, UDINT_TO_STRING(stEvent.nIdentity));
sXmlContent := CONCAT(sXmlContent, '</ID>');
```

```
sXmlContent := CONCAT(sXmlContent, '<Text>');
sXmlContent := CONCAT(sXmlContent, stEvent.sText);
sXmlContent := CONCAT(sXmlContent, '</Text>');
sXmlContent := CONCAT(sXmlContent, '</Event>');

// Write to file (simplified - actual implementation would be more complex)
// fbFileWrite(sFilePath := sFilePath, sContent := sXmlContent);

StoreEvent := TRUE;
```

```
// Example Storage Implementation: FB_MockStorage
// Implements I_PersistentEventStorage for unit testing
// Does NOT actually store anything - just records that it was called
FUNCTION_BLOCK FB_MockStorage IMPLEMENTS I_PersistentEventStorage
VAR
    nStoreEventCallCount : UDINT := 0;  // Count method calls
    stLastStoredEvent    : ST_Event;     // Record last event
    bSimulateFailure     : BOOL := FALSE; // For testing error handling
END_VAR
```

```
// Mock implementation of StoreEvent
METHOD StoreEvent : BOOL
VAR_INPUT
    stEvent : REFERENCE TO ST_Event;
END_VAR

// Record that this method was called (for test verification)
nStoreEventCallCount := nStoreEventCallCount + 1;
stLastStoredEvent := stEvent;

// Optionally simulate failure for testing error handling
IF bSimulateFailure THEN
    StoreEvent := FALSE;
ELSE
    StoreEvent := TRUE;
END_IF
```

Quick Reference Card

Interface Declaration Syntax

```
INTERFACE <InterfaceName>

// Method signatures (no implementation)
METHOD <MethodName> : <ReturnType>
VAR_INPUT
    <param> : <DataType>;
END_VAR
```

Implementing Interfaces Syntax

```
// Single interface
FUNCTION_BLOCK <FBName> IMPLEMENTS <InterfaceName>

// Multiple interfaces
FUNCTION_BLOCK <FBName> IMPLEMENTS <Interface1>, <Interface2>, <
    Interface3>

// Combined with inheritance
FUNCTION_BLOCK <FBName> EXTENDS <ParentFB> IMPLEMENTS <Interface1>, <
    Interface2>
```

Dependency Injection via FB_init

```
METHOD FB_init : BOOL
VAR_INPUT
    bInitRetains : BOOL;
    bInCopyCode  : BOOL;
    iDependency  : <InterfaceType>;  // Injected dependency
END_VAR

// Store the dependency
THIS^.iLocalReference := iDependency;
```

OOP Keywords Summary

Keyword	Purpose
`INTERFACE`	Define an interface contract
`END_INTERFACE`	End interface definition
`IMPLEMENTS`	Declare that FB follows an interface
`EXTENDS`	Inherit from parent FB
`FB_init`	Constructor method for dependency injection
`THIS^`	Reference to current instance

Table 3: Interface-Related Keywords

Common Naming Conventions

Prefix	Type	Example
`I_`	Interface	`I_LightTower`, `I_Storage`
`FB_`	Function Block	`FB_EventLogger`
`i`	Interface variable	`iActiveTower`, `iStorage`
`fb`	FB instance	`fbXmlStorage`

Table 4: Naming Conventions for Interfaces

Last Updated: March 14, 2026